

Guía de Aprendizaje: Creación de una Matriz Dinámica

Malak Sanchez

Workshop 2026-II

21 de agosto de 2026

1. El reto: ¿Qué es una matriz dinámica?

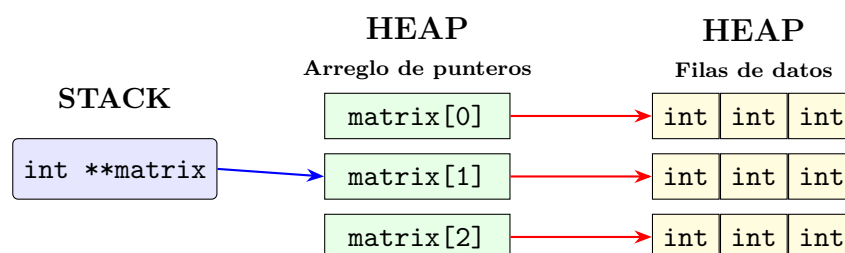
En C, una matriz convencional puede declararse como `int mat[F][C]`. Cuando sus dimensiones se conocen durante la ejecución, necesitamos reservar memoria dinámicamente. Esta guía usa una representación con **puntero a punteros** (`int **`).

2. Estructura Mental

Imagina una matriz dinámica no como un bloque sólido, sino como:

1. Un puntero principal (`int **matriz`).
2. Un arreglo de **punteros** (uno por cada fila).
3. Cada uno de esos punteros apunta a un arreglo de **enteros** (las columnas).

3. Dibujo Ilustrativo



4. Cómo se construye (El Código)

Debemos hacer la reserva en **capas**. Primero las filas, luego cada fila individual.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void){
5      int rows = 3, cols = 4;
6
7      int **matrix = malloc(rows * sizeof *matrix);
8      if (matrix == NULL) {
9          printf("No se pudo reservar memoria.\n");
10         return 1;
11     }
12
13     for (int i = 0; i < rows; i++) {
14         matrix[i] = malloc(cols * sizeof *matrix[i]);
15         if (matrix[i] == NULL) {
16             while (i > 0) {
17                 free(matrix[--i]);
18             }
19             free(matrix);
20             printf("No se pudo reservar memoria.\n");
21             return 1;
22         }
23     }
24
25     matrix[1][2] = 50;
26
27     for (int i = 0; i < rows; i++) {
28         free(matrix[i]);
29     }
30     free(matrix);
31
32     return 0;
33 }
```

5. Regla de Oro

Por cada malloc que hagas, debe existir un free. En una matriz de N filas, haces $N + 1$ allocations (malloc): una para el arreglo de punteros y N para las filas datos. Por lo tanto, ¡necesitas $N + 1$ llamadas a free!

6. Errores comunes

- **Liberar solo el padre:** Si haces `free(matrix)` sin liberar las filas primero, habrás perdido la dirección de las filas y esa memoria quedará "bloqueada" (Memory Leak).
- **Confusión en sizeof:** En el primer malloc se usa `sizeof(int *)`, porque estamos guardando punteros. En el segundo se usa `sizeof(int)`, porque guardamos enteros.